

programming

Credit Hours: 3-1

Course Outline

Programming fundamentals are the core concepts that every developer should learn before moving into advanced software development. They begin with understanding what programming is, how software works, and how programming languages communicate with computers. A programmer first learns basic syntax, variables, data types, input/output operations, and operators used for calculations and logical comparisons. After this, conditional statements such as ``if``, ``else``, and ``elif`` are introduced to control program flow and decision-making. Loops like ``for`` and ``while`` help automate repetitive tasks, while functions allow code reusability and modular design. Students then learn about important data structures such as arrays, lists, dictionaries, tuples, stacks, and queues, along with string manipulation techniques. Object-oriented programming concepts including classes, objects, inheritance, encapsulation, and polymorphism are essential for building scalable applications. Error handling and debugging teach developers how to identify and fix problems in code efficiently. File handling enables programs to read and write data from files, while algorithms and problem-solving techniques improve logical thinking and efficiency. Searching and sorting algorithms introduce optimization concepts and computational thinking. Understanding memory management, stack vs heap, and execution flow helps programmers write better-performing applications. Modern programming fundamentals also include version control systems like Git and GitHub, database basics such as SQL and CRUD operations, API concepts, and software development best practices including clean code and testing. Together, these concepts create the foundation required to build real-world applications in web development, mobile apps, artificial intelligence, backend systems, and other software engineering fields.

16-Week Course Plan

Week 1

****Week 1: Introduction to Programming Fundamentals****

****3-1****

Lecture 1: What is Programming?

Learning Objectives

- * Define programming and its importance
- * Identify the fundamental concepts of programming

Concepts

Programming is the process of designing, writing, testing, debugging, and maintaining the source code of computer programs. It involves creating a set of instructions that a computer can execute to perform a specific task. Programming fundamentals include syntax, data types, variables, control structures, functions, and object-oriented programming concepts.

Lecture 2: Software Development and Programming Languages

Learning Objectives

- * Understand the software development process
- * Identify the role of programming languages in software development

Concepts

Software development involves several stages, including planning, design, coding, testing, and deployment. Programming languages are used to write code that instructs the computer to perform specific tasks. Popular programming languages include Python, Java, C++, and JavaScript.

Lecture 3: Basic Computer Concepts

Learning Objectives

- * Understand basic computer concepts
- * Identify the components of a computer system

Concepts

A computer system consists of hardware and software components. The hardware includes the central processing unit (CPU), memory (RAM), storage devices, and input/output devices. The software includes the operating system, programming languages, and application software.

Lab 1: Introduction to Programming Editors

Learning Objectives

- * Familiarize with a programming editor
- * Practice basic code writing and execution

Concepts

In this lab, students will practice writing and executing basic code using a programming editor. They will learn how to save files, execute code, and use basic syntax.

Week 2

****Week 2: Variables, Data Types, and Operators****

****3-1****

Lecture 1: Variables and Data Types

Learning Objectives:

- Define variables and data types in programming.
- Identify the basic data types in a programming language.

Concepts:

Variables are containers that hold values in a program. Data types define the type of value a variable can hold. In most programming languages, basic data types include integers, floats, strings, and booleans.

Lecture 2: Operators for Calculations and Comparisons

Learning Objectives:

- Understand arithmetic operators for calculations.
- Learn comparison operators for logical decisions.

Concepts:

Arithmetic operators (+, -, *, /, etc.) are used for calculations. Comparison operators (==, !=, >, <, etc.) are used for logical comparisons in conditional statements.

Lecture 3: Input/Output Operations

Learning Objectives:

- Understand how to read user input from the keyboard.
- Learn how to display output on the screen.

Concepts:

Input operations allow users to provide data to the program. Output operations display results to the user. Common input/output functions include ``print()`` and ``input()``.

****Lab 1: Variables, Data Types, and Operators****

- Write a program to declare variables of different data types.
- Perform arithmetic operations using operators.

- Use comparison operators in conditional statements.
- Read user input and display output on the screen.

Week 3

****Week 3: Conditional Statements and Functions****

****3-1****

Lecture 1: Conditional Statements

Learning Objectives

- Understand the concept of conditional statements
- Learn to use `if`, `else`, and `elif` statements
- Apply conditional statements in programming

Concepts

Conditional statements are used to control program flow based on conditions. `if` statements are used to execute a block of code if a condition is true. `else` statements are used to execute a block of code if the condition is false. `elif` statements are used to check another condition if the first condition is false.

Lecture 2: Conditional Statements (continued)

Learning Objectives

- Understand the syntax of conditional statements
- Learn to use logical operators in conditional statements
- Apply conditional statements with logical operators

Concepts

Logical operators `and`, `or`, and `not` are used to combine conditions in conditional statements. The `in` operator is used to check if a value is in a list or a string.

Lecture 3: Functions

Learning Objectives

- Understand the concept of functions
- Learn to define and call functions
- Apply functions to write reusable code

Concepts

Functions are reusable blocks of code that can be called multiple times in a program. Functions take arguments and return values. Functions can be used to organize code and make it more readable.

****Lab 1: Practice Conditional Statements and Functions****

- * Practice writing conditional statements using ``if``, ``else``, and ``elif``
- * Practice defining and calling functions
- * Apply functions to write reusable code

Week 4

****Week 4: Control Structures (3-1)****

Lecture 1: Conditional Statements (≈75 words)

Conditional Statements

Learning Objectives

- Understand the use of ``if``, ``else``, and ``elif`` statements
- Learn how to write conditional statements in programming

Concepts

- Conditional statements: ``if``, ``else``, and ``elif``
- Syntax: ``if condition:``, ``else:``, ``elif condition:``
- Example: Basic conditional statement

Lecture 2: Loop Control (≈75 words)

Loop Control

Learning Objectives

- Understand the use of ``for`` and ``while`` loops
- Learn how to write loop control statements in programming

Concepts

- Loops: ``for`` and ``while``
- Syntax: ``for variable in iterable:``, ``while condition:``
- Example: Basic ``for`` loop

Lecture 3: Loop Examples (≈75 words)

Loop Examples

Learning Objectives

- Understand how to use loops in real-world scenarios
- Learn how to troubleshoot loop-related issues

Concepts

- Loop examples: printing numbers, iterating over strings

- Common loop-related issues: infinite loops, missing iterations
- Example: Real-world loop example

Lab 1: Loop Practice (≈30 words)

Lab 1: Loop Practice

Learning Objectives

- Practice writing loop control statements
- Troubleshoot loop-related issues

Concepts

- Write a `for` loop to print numbers from 1 to 10
- Write a `while` loop to iterate over a string

Week 5

****Week 5: Control Structures and Functions****

****3-1****

Lecture 1: Conditional Statements

Conditional Statements

Conditional statements are used to control program flow based on conditions or decisions. They help programmers write efficient and effective code.

Learning Objectives

- * Understand the concept of conditional statements
- * Learn to use `if`, `else`, and `elif` statements
- * Apply conditional statements in programming scenarios

Concepts

- * Conditional statements: `if`, `else`, `elif`
- * Conditional statement syntax
- * Examples of conditional statements in programming

Lecture 2: Loop Control

Loop Control

Loops are used to automate repetitive tasks in programming. They help programmers write efficient and effective code.

Learning Objectives

- * Understand the concept of loop control
- * Learn to use `for` and `while` loops
- * Apply loop control in programming scenarios

Concepts

- * Loop control: `for`, `while`
- * Loop syntax and examples
- * Loop termination conditions

Lecture 3: Functions

Functions

Functions are reusable blocks of code that perform specific tasks. They help programmers write efficient and modular code.

Learning Objectives

- * Understand the concept of functions
- * Learn to define and call functions
- * Apply functions in programming scenarios

Concepts

- * Function syntax and definition
- * Function arguments and return types
- * Function examples and applications

Lab 1: Control Structures and Functions

- * Practice writing conditional statements, loops, and functions in a programming language of choice
- * Apply control structures and functions in a programming scenario or project

Week 6

****Week 6: Control Structures and Functions****

Lecture 1: Conditional Statements (~75 words)

Conditional Statements

Learning Objectives

- * Understand the concept of conditional statements
- * Learn to use `if`, `else`, and `elif` statements

Concepts

Conditional statements are used to control program flow based on conditions. `if` statements execute code when a condition is true, `else` statements execute code when a condition is false, and `elif` statements check multiple conditions.

Lecture 2: Loops (≈75 words)

Loops

Learning Objectives

- * Understand the concept of loops
- * Learn to use `for` and `while` loops

Concepts

Loops are used to automate repetitive tasks. `for` loops iterate over a sequence, and `while` loops execute code while a condition is true.

Lecture 3: Functions (≈75 words)

Functions

Learning Objectives

- * Understand the concept of functions
- * Learn to define and call functions

Concepts

Functions are reusable blocks of code that perform a specific task. They help modularize code and improve readability.

Lab 1: Control Structures and Functions (≈30 words)

Lab 1: Control Structures and Functions

Learning Objectives

- * Practice using conditional statements, loops, and functions
- * Apply control structures and functions to solve problems

Concepts

This lab will provide hands-on experience with control structures and functions. Students will practice using `if`, `else`, `elif`, `for`, and `while` statements, as well as defining and calling functions.

Week 7

****Week 7: Functions and Modules (3-1)****

Lecture 1: Introduction to Functions

Functions are blocks of code that perform a specific task. They help in code reusability and modular design. Functions can take arguments and return values. In Python, functions are defined using the `def` keyword.

Lecture 2: Function Arguments and Return Types

Functions can have multiple arguments and return types. Argument types can be specified using type hints. Return types can be specified using the `->` operator. Functions can also have default argument values.

Lecture 3: Modules and Importing Functions

Modules are files that contain functions and variables. Modules can be imported into a program using the `import` statement. Functions from a module can be imported using the `from` keyword. Modules can be used to organize code and make it reusable.

Lab 1: Creating and Using Functions

Create a Python program that uses functions to perform calculations. Define a function that takes two arguments and returns their sum. Use the function in the program to calculate the sum of two numbers.

Week 8

Week 8: Loops and Functions (3-1)

Lecture 1: Introduction to Loops (60 minutes)

Looping Statements

Loops allow us to automate repetitive tasks by executing a block of code multiple times. There are two primary types of loops: `for` and `while`.

Learning Objectives:

- Understand the difference between `for` and `while` loops.
- Learn how to use `for` loops to iterate over sequences.
- Learn how to use `while` loops for repetitive tasks.

Concepts:

- **For Loops:** `for` loop iterates over a sequence (e.g., list, tuple, string) and executes the code inside the loop for each item.
- **While Loops:** `while` loop executes the code as long as a certain condition is true.

Lecture 2: Advanced Loop Concepts (60 minutes)

Loop Control and Break/Continue Statements

We can control the flow of a loop using `break` and `continue` statements. `break` exits the loop entirely, while `continue` skips to the next iteration.

Learning Objectives:

- Understand how to use break and continue statements to control loop flow.
- Learn how to use nested loops to iterate over multiple sequences.

Concepts:

- **Break Statement:** Terminates the loop and continues execution at the next statement.
- **Continue Statement:** Skips to the next iteration of the loop.

Lecture 3: Functions (60 minutes)

Introduction to Functions

Functions allow us to reuse code by grouping a set of statements together and giving them a name.

Learning Objectives:

- Understand the benefits of using functions in programming.
- Learn how to define and call functions in a program.

Concepts:

- **Function Definition:** A function is defined using the `def` keyword followed by the function name and parameters.
- **Function Call:** A function is called by specifying the function name followed by arguments.

Lab 1: Practice Loops and Functions (120 minutes)

Lab Exercises

- Write a program that uses a for loop to iterate over a list of numbers and print each number.
- Write a program that uses a while loop to ask the user for input until they enter a specific value.
- Write a program that defines a function to calculate the area of a rectangle and calls the function with different dimensions.

Week 9

Week 9: Object-Oriented Programming (OOP) Concepts

3-1

Lecture 1: Introduction to Classes and Objects

Classes and objects are fundamental concepts in object-oriented programming. A class defines a blueprint for creating objects, which have their own set of attributes (data) and methods (functions).

*** Learning Objectives:**

- + Define classes and objects in OOP
- + Identify the characteristics of classes and objects

*** Concepts:**

- + Class: a template for creating objects
- + Object: an instance of a class
- + Attributes: data associated with an object
- + Methods: functions that operate on an object

Lecture 2: Inheritance and Encapsulation

Inheritance and encapsulation are two key principles of OOP.

* Learning Objectives:

- + Explain inheritance and its types (single, multiple, multilevel)
- + Describe encapsulation and its benefits

* Concepts:

- + Inheritance: a mechanism for creating a new class based on an existing class
- + Single inheritance: one class inherits from another
- + Multiple inheritance: a class inherits from multiple classes
- + Multilevel inheritance: a class inherits from a class that inherits from another class
- + Encapsulation: bundling data and methods that operate on that data within a single unit

Lecture 3: Polymorphism and Interfaces

Polymorphism and interfaces are essential concepts in OOP.

* Learning Objectives:

- + Explain polymorphism and its types (method overriding, method overloading)
- + Describe interfaces and their benefits

* Concepts:

- + Polymorphism: the ability of an object to take on multiple forms
- + Method overriding: a method in a subclass has the same name, return type, and parameters as a method in its superclass
- + Method overloading: multiple methods with the same name but different parameters
- + Interface: a abstract class that defines a contract that must be implemented by any class that implements it

Lab 1: Implementing OOP Concepts

Implement the OOP concepts learned in lectures using a programming language of your choice. Create a simple program that demonstrates classes, objects, inheritance, encapsulation, polymorphism, and interfaces.

* Learning Objectives:

- + Apply OOP concepts to a practical problem
- + Demonstrate understanding of OOP principles
- * Concepts:
 - + Implement classes, objects, inheritance, encapsulation, polymorphism, and interfaces in a programming language
- + Use a programming language to demonstrate OOP concepts

Week 10

****Week 10: Object-Oriented Programming (OOP) Concepts****

****Lecture 1: Introduction to Classes and Objects (≈150 words)****

Learning Objectives

- * Understand the concept of classes and objects in OOP
- * Learn how to define and create classes and objects

Concepts

A class is a blueprint or template that defines the properties and behavior of an object. An object is an instance of a class, with its own set of attributes (data) and methods (functions). Classes and objects are the fundamental building blocks of OOP. Classes define the structure and behavior of objects, while objects represent individual instances of those classes.

****Lecture 2: Inheritance and Encapsulation (≈150 words)****

Learning Objectives

- * Understand the concept of inheritance and encapsulation in OOP
- * Learn how to implement inheritance and encapsulation in programming

Concepts

Inheritance is the mechanism by which one class can inherit the properties and behavior of another class. Encapsulation is the concept of bundling data and methods that operate on that data within a single unit, such as a class or object. Inheritance allows for code reuse and promotes modularity, while encapsulation helps to hide implementation details and improve code organization.

****Lab: Implementing Classes, Objects, Inheritance, and Encapsulation (1 hour)****

- * Implement a simple class and object in a programming language of your choice
- * Use inheritance to create a new class that inherits properties and behavior from a parent class
- * Encapsulate data and methods within a class or object to demonstrate encapsulation

****Total Lecture Hours: 3 | Total Lab Hours: 1****

Week 11

****Week 11: Functions and Modular Design (3-1)****

Lecture 1: Introduction to Functions (Lecture 1 of 3)

Functions are blocks of code that perform a specific task and can be reused throughout a program. They help in modular design, making code more organized and easier to maintain.

****Learning Objectives:****

- Define functions and their importance in programming
- Explain the benefits of modular design

****Concepts:****

- Function syntax and structure
- Function parameters and return types
- Function examples: mathematical operations, string manipulation, and input/output operations

Lecture 2: Function Arguments and Return Values (Lecture 2 of 3)

Functions can take arguments and return values, which are crucial for their functionality and reusability.

****Learning Objectives:****

- Explain function arguments and their types
- Describe the concept of return values and their types

****Concepts:****

- Function arguments: passing variables and data types
- Return values: basic syntax and examples

Lecture 3: Function Examples and Best Practices (Lecture 3 of 3)

Functions can be used to simplify complex tasks and improve code readability.

****Learning Objectives:****

- Provide examples of function usage in real-world scenarios
- Discuss best practices for writing functions

****Concepts:****

- Function examples: calculating area and perimeter of a rectangle
- Function best practices: naming conventions, commenting, and testing

Lab 1: Implementing Functions in a Program

Implement a simple calculator program using functions for addition, subtraction, multiplication, and division.

****Learning Objectives:****

- Apply function concepts to a real-world scenario

- Practice modular design and code reusability

****Deliverables:****

- A working calculator program with functions for basic arithmetic operations.

Week 12

****Week 12: Error Handling and Debugging****

****3-1****

Lecture 1: Introduction to Error Handling

Learning Objectives

- Understand the importance of error handling in programming
- Learn how to identify and handle common errors
- Implement basic error handling techniques in code

Concepts

Error handling is the process of anticipating and managing errors that occur during the execution of a program. It involves identifying potential errors, handling them, and preventing the program from crashing. Common errors include syntax errors, runtime errors, and logical errors.

Lecture 2: Debugging Techniques

Learning Objectives

- Understand the importance of debugging in programming
- Learn various debugging techniques, including print statements, debuggers, and logging
- Implement debugging techniques in code

Concepts

Debugging is the process of identifying and fixing errors in a program. Techniques include print statements, debuggers, and logging. Debuggers allow developers to step through code, inspect variables, and set breakpoints.

Lecture 3: Exception Handling

Learning Objectives

- Understand how to use try-except blocks to handle exceptions
- Learn how to catch and handle specific exceptions
- Implement exception handling in code

Concepts

Try-except blocks are used to handle exceptions in code. The try block contains code that may raise an exception, and the except block contains code that handles the exception. Specific

exceptions can be caught and handled using the except keyword.

****Lab 1: Implementing Error Handling and Debugging****

- Implement error handling and debugging techniques in a provided code snippet
- Identify and fix common errors in the code
- Use a debugger to step through the code and identify issues

Week 13

****Week 13: Object-Oriented Programming (OOP) Concepts****

Lecture 1: Classes and Objects (3-1)

Learning Objectives

- Define classes and objects in OOP.
- Explain the relationship between classes and objects.
- Create classes and objects in a programming language.

Concepts

A class is a blueprint or template that defines the properties and behavior of an object. An object is an instance of a class, which has its own set of attributes (data) and methods (functions). Classes and objects are used to model real-world entities and interactions. In programming, classes are defined using keywords such as `class`, `extends`, and `implements`, while objects are created using the `new` keyword. Understanding classes and objects is essential for building scalable and maintainable software applications.

Lecture 2: Inheritance and Encapsulation (3-1)

Learning Objectives

- Explain inheritance and its types (single, multiple, and multilevel).
- Define encapsulation and its importance in OOP.
- Use inheritance and encapsulation in a programming language.

Concepts

Inheritance is a mechanism that allows one class to inherit the properties and behavior of another class. There are three types of inheritance: single (one class inherits from another), multiple (one class inherits from multiple classes), and multilevel (one class inherits from another class, which inherits from a third class). Encapsulation is the practice of hiding the implementation details of an object from the outside world and only exposing the necessary information through public methods. Inheritance and encapsulation are used to promote code reuse, modularity, and maintainability.

Lecture 3: Polymorphism and Abstraction (3-1)

Learning Objectives

- Explain polymorphism and its types (method overriding and method overloading).

- Define abstraction and its importance in OOP.
- Use polymorphism and abstraction in a programming language.

Concepts

Polymorphism is the ability of an object to take on multiple forms, depending on the context in which it is used. There are two types of polymorphism: method overriding (a subclass provides a different implementation of a method already defined in its superclass) and method overloading (multiple methods with the same name but different parameters). Abstraction is the practice of exposing only the necessary information to the outside world while hiding the implementation details.

Polymorphism and abstraction are used to promote flexibility, modularity, and reusability in software applications.

Lab 1: OOP Concepts Implementation (1-1)

Learning Objectives

- Implement classes and objects in a programming language.
- Use inheritance and encapsulation in a programming language.
- Implement polymorphism and abstraction in a programming language.

Concepts

In this lab, students will implement the OOP concepts learned in the lectures using a programming language such as Java or Python. They will create classes and objects, use inheritance and encapsulation, and implement polymorphism and abstraction. The lab will help students understand the practical application of OOP concepts in software development.

Week 14

****Week 14: Loops and Iteration****

****3-1****

Lecture 1: Introduction to Loops

*** **Learning Objectives:****

- + Understand the concept of loops in programming
- + Learn the syntax of for and while loops

*** **Concepts:****

For loops are used to execute a block of code repeatedly for a specified number of times. The syntax is: `for variable in iterable: code_block``. While loops are used to execute a block of code as long as a condition is true. The syntax is: `while condition: code_block``.

Lecture 2: For Loops

*** **Learning Objectives:****

- + Learn how to use for loops to iterate over lists and tuples
- + Understand how to use the `range()` function to create a sequence of numbers

* **Concepts:**

For loops can be used to iterate over lists and tuples. The `range()` function can be used to create a sequence of numbers that can be used in the loop. Example: `for i in range(10): print(i)`.

Lecture 3: While Loops

* **Learning Objectives:**

- + Learn how to use while loops to execute a block of code repeatedly
- + Understand how to use the `break` and `continue` statements to control the loop

* **Concepts:**

While loops can be used to execute a block of code repeatedly as long as a condition is true. The `break` statement can be used to exit the loop prematurely, while the `continue` statement can be used to skip the current iteration.

Lab 1: Looping Exercises

- * Implement a for loop to print the numbers from 1 to 10.
- * Implement a while loop to ask the user for their name until they enter a valid name.
- * Modify the previous exercise to use a for loop instead of a while loop.

Week 15

Week 15: Object-Oriented Programming (OOP) and Encapsulation

3-1

Lecture 1: Introduction to OOP

Learning Objectives

- * Define Object-Oriented Programming (OOP)
- * Explain the four pillars of OOP: Encapsulation, Abstraction, Inheritance, and Polymorphism

Concepts

Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. OOP is based on the idea of encapsulation, which binds together data and methods that manipulate that data. The four pillars of OOP are:

- * Encapsulation: Hiding internal implementation details and exposing only necessary information to the outside world.
- * Abstraction: Representing complex systems in a simplified manner.
- * Inheritance: Creating new classes based on existing classes.
- * Polymorphism: The ability of an object to take on multiple forms.

Lecture 2: Encapsulation and Classes

Learning Objectives

- * Explain encapsulation and its importance in OOP
- * Define a class and its components
- * Identify the benefits of using classes

Concepts

Encapsulation is the concept of bundling data and methods that operate on that data within a single unit. In OOP, this unit is called a class. A class is a blueprint for creating objects. The benefits of using classes include:

- * Data hiding: Protecting internal data from external interference.
- * Code reusability: Reducing code duplication by creating reusable classes.
- * Easier maintenance: Modifying class behavior without affecting other parts of the program.

Lecture 3: Inheritance and Polymorphism

Learning Objectives

- * Explain inheritance and its types
- * Define polymorphism and its types
- * Provide examples of inheritance and polymorphism in programming

Concepts

Inheritance is the process of creating a new class based on an existing class. There are two types of inheritance: single inheritance and multiple inheritance. Polymorphism is the ability of an object to take on multiple forms. There are two types of polymorphism: method overloading and method overriding. Inheritance and polymorphism enable code reusability, flexibility, and extensibility.

Lab 1: Implementing OOP Concepts

Learning Objectives

- * Implement encapsulation, inheritance, and polymorphism in a programming language
- * Write a program that demonstrates OOP concepts
- * Analyze the benefits of using OOP in software development

Concepts

In this lab, students will implement OOP concepts in a programming language of their choice. They will create classes, implement inheritance, and demonstrate polymorphism. The lab will provide hands-on experience with OOP concepts and their benefits in software development.

Week 16

****Week 16: Version Control Systems and Database Basics****

****3-1****

Lecture 1: Introduction to Version Control Systems

Git and GitHub Basics

Git is a version control system that helps developers track changes in their code. GitHub is a web-based platform that hosts Git repositories. Learning Git and GitHub is essential for collaboration and code management.

Learning Objectives:

- Understand the basics of Git and GitHub
- Learn how to create a Git repository and commit changes
- Understand the concept of branching and merging

Concepts:

- Git commands: ``git add``, ``git commit``, ``git push``, ``git pull``
- GitHub features: repositories, branches, commits, pull requests

Lecture 2: Database Basics

Introduction to SQL and CRUD Operations

SQL (Structured Query Language) is a language used to manage and manipulate data in relational databases. CRUD (Create, Read, Update, Delete) operations are essential for database management.

Learning Objectives:

- Understand the basics of SQL and database management
- Learn how to perform CRUD operations using SQL
- Understand the concept of database normalization

Concepts:

- SQL syntax: ``SELECT``, ``INSERT``, ``UPDATE``, ``DELETE``
- Database normalization: first normal form, second normal form, third normal form

Lecture 3: API Concepts and Software Development Best Practices

Introduction to APIs and Clean Code

APIs (Application Programming Interfaces) enable different applications to communicate with each other. Clean code and testing are essential for software development best practices.

Learning Objectives:

- Understand the basics of APIs and API design
- Learn how to write clean code and follow software development best practices
- Understand the importance of testing in software development

Concepts:

- API concepts: RESTful APIs, API endpoints, API requests and responses
- Clean code principles: single responsibility principle, don't repeat yourself, etc.
- Testing concepts: unit testing, integration testing, etc.

Lab 1: Version Control Systems and Database Basics

Practical Exercise: [Git and GitHub](#)

- Create a Git repository and commit changes
- Perform CRUD operations using SQL
- Design a simple API using RESTful principles

Note: The lab exercise should be specific and focused on the concepts learned in the lectures.